Learning to Place Guards by Reinforcement: A Geo-Free Neural Policy for the Vertex-Guard Art Gallery Problem

Domagoj Ševerdija Jurica Maltar Nathan Chappel
Domagoj Matijević

Abstract

We ask what a neural policy internalizes about the vertex-guard Art Gallery Problem when it is trained from a coverage-aware reward over its own rollouts and denied any geometric oracle at inference. The policy is an LSTM pointer network trained with preference optimization on Bradley–Terry pairs of self-generated rollouts; at test time it sees only vertex coordinates — no visibility matrix, no per-vertex area feature, no oracle call. To separate what the learner represents from what its decoder expresses, we freeze the trained encoder and train a small single-shot per-vertex classifier over its embeddings, again with no geometric input at inference. This probe closes the in-distribution feasibility tail on the displayed seed and compresses the out-of-distribution feasibility tail across four probe seeds on polygons whose vertex counts reach roughly seven times the training range, from 310/2107 under-covered polygons under the policy alone to 17.5 ± 12.5 on average across seeds — a reduction of roughly an order of magnitude. We read this as evidence that the reinforcement-trained encoder already carries the geometric structure a feasibility decision needs, and that the policy's residual tail is a decoder-calibration limit rather than a representation limit. The cost of recovering feasibility — a controlled rise in guard count — is reported throughout.

1 Introduction

The Art Gallery Problem (AGP) asks for the minimum number of guards needed to observe every point of a simple polygon [10, 12], with applications in surveillance, sensor placement, and robotics [6, 3]. We focus on the *vertex-guard* variant, in which guards must be chosen from the polygon's vertices; the problem remains NP-hard and is a discrete instance of geometric set cover with non-uniform, polygon-dependent visibility regions. The question this paper investigates is whether a neural policy can learn to place such guards from coordinates alone, trained from a coverage-aware reward signal over its own rollouts, with no visibility matrix and no geometric oracle at the moment it chooses guards. We are not asking whether a learned solver can match a classical greedy heuristic on

guard count — it cannot, and we do not claim it does. We are asking whether the underlying combinatorial-geometric task is learnable when the model is denied any explicit geometric input at test time.

The reinforcement method we evaluate is an LSTM pointer-network policy trained with Preference Optimization (PO) [13] on Bradley–Terry preference pairs [4] drawn from the policy's own rollouts. The policy emits a vertex-index sequence with an EOS token; the reward couples coverage and guard usage. PO replaces REINFORCE advantages [17, 1] with binary preferences between pairs of rollouts, sidestepping REINFORCE's variance and the brittleness of supervised cloning of search labels. We refer to inference under this policy as *geo-free*: at test time the network sees only the polygon's vertex coordinates, with no precomputed visibility matrix, no CGAL output, and no per-vertex area feature.

The policy works, to a degree. On a held-out in-distribution split its guard sets are competitive in cardinality with classical baselines, but the per-polygon coverage distribution has a tail: a non-trivial fraction of polygons end up below the feasibility line one would want a deployed solver to clear, and the failure rate grows on out-of-distribution polygons whose vertex counts exceed the training range. The reinforcement method, by itself, is not feasibility-guaranteed.

To check whether this residual reflects a true limit of the reinforcement-learned representation or only a calibration problem in the decoder, we train a small per-vertex classifier that reads only the frozen encoder embeddings of the policy and predicts a vertex-inclusion probability, controlled by a single scalar threshold. The classifier has no access to visibility information at inference, and it compresses the feasibility tail substantially both in and out of distribution. We read this as evidence that the reinforcement-trained encoder already carries enough geometric structure to support a feasibility decision, and that the policy's coverage tail was a decoder-calibration limit rather than a representation limit. The cost of recovering feasibility is a controlled rise in guard count, which we report explicitly.

Contributions. This paper makes four contributions.

- **C1 A representation–decoder decomposition.** Freezing the reinforcement-trained encoder and training a single-shot per-vertex probe over its embeddings isolates what the learner *represents* from what its decoder *expresses*. The pattern is methodological and not specific to AGP: it turns "does the model know X" into a measurable question about a frozen representation.

- **C2 Out-of-distribution generalization at scale.** The probe largely recovers feasibility on polygons whose vertex counts reach roughly seven times the training range, cutting the rate of under-covered polygons from 310/2107 to 17.5 ± 12.5 on average across four probe seeds (Section 6.4).

- **C3 Geo-free inference as a protocol.** Restricting test-time inputs to vertex coordinates isolates what a learner internalizes from what a visibility

oracle could compute for it, making "no explicit geometric knowledge" a measurable property rather than a rhetorical one (Section 2.4).

C4 A structural finding on post-processing. Iterative add/remove/stop editors fail to improve a strong seed in any coverage-preserving regime, whereas the single-shot probe is empirically a fixed point under iteration (Section 6.6); the contrast is what makes the single-shot design the one that exposes the encoder's information.

We do not claim to beat classical greedy or local-search heuristics on guard count; we claim, and document, that vertex-guard placement is learnable to a measurable degree by a reinforcement method that never reads an explicit visibility object at inference.

The remainder of the paper is organized as follows. Section 2 fixes notation and the geo-free constraint. Section 3 reviews related work. Section 4 describes the reinforcement method, what it does and does not absorb, and the representation probe. Section 5 describes the experimental setup. Section 6 reports the results. Section 7 reads the results against the question. Section 8 states limitations and Section 9 concludes.

2 Problem and Notation

2.1 Polygons and visibility

Let $s \subset \mathbb{R}^2$ denote a simple polygonal region (the closed set consisting of its interior and boundary), with n vertices at coordinates $x_1, \ldots, x_n \in \mathbb{R}^2$ and vertex index set $V(s) = \{1, \ldots, n\}$. A *guard* is any point $g \in s$ with unobstructed 360° visibility inside s. For a finite guard set $G \subseteq s$, the visibility region and area-normalized coverage are

$$\mathrm{Vis}(G) \;=\; \{\, q \in s : \exists g \in G,\ \overline{gq} \subseteq s \,\}, \qquad \mathrm{Cov}(G \mid s) \;=\; \frac{\mathrm{Area}(\mathrm{Vis}(G))}{\mathrm{Area}(s)} \in [0,1]. \tag{1}$$

We write $\mathrm{Cov}(G)$ throughout when s is clear from context.

2.2 The vertex-guard variant

In the *vertex-guard Art Gallery Problem (AGP)* guards are restricted to the polygon's vertices: $S \subseteq V(s)$, with physical positions $\{x_i : i \in S\}$. The coverage of S is $\mathrm{Cov}(S)$, with visibility region $\mathrm{Vis}(S) = \{q \in s : \exists i \in S,\ \overline{x_i q} \subseteq s\}$. We define the *vertex-guard optimum* directly:

$$\mathrm{OPT}(s) \;=\; \min\big\{|S| : S \subseteq V(s),\ \mathrm{Cov}(S) = 1\big\}. \tag{2}$$

Restricting guards to $V(s)$ reduces placement to a finite discrete choice over n vertices — the natural action space for a pointer-network policy [16, 1, 8, 9] — while retaining the full NP-hardness and non-uniform visibility structure of

the problem. The public instance library provides *point-guard* solutions (guards anywhere in s); these are not the vertex-guard optimum and are not used as OPT here (see Section 5.1).

2.3 Evaluation metrics

Let N denote the cardinality of the evaluation partition under discussion. Mean coverage $\frac{1}{N}\sum \mathrm{Cov}(S)$ summarizes overall performance, but the per-polygon distribution behind the mean is at least as informative: we report the counts $\#\{\mathrm{Cov}(S) \geq c\}$ for threshold values $c \in \{0.95, 0.99, 0.999, 1\}$, and the worst-case polygon coverage $\min_s \mathrm{Cov}(S)$. Note that c is distinct from the training-time coverage gate $\tau = 0.99$ used to label local-search targets (Section 4.4). Solution compactness is measured by two ratios: the normalized guard count $|S|/n$, which is the guard set size relative to the polygon's vertex count, and the approximation ratio $|S|/\mathrm{OPT}$, relative to the vertex-guard optimum. For binomial count proportions we report Wilson 95% confidence intervals on $\#\{\mathrm{Cov}(S) \geq c\}/N$.

2.4 Geo-free inference

We will say that an inference procedure is *geo-free* if its inputs at test time are limited to the polygon's vertex coordinates and learned features derived from them — no polygon visibility matrix, no CGAL-computed visibility regions, no per-vertex area or visibility-fraction feature. Geo-free inference does not preclude using exact visibility during evaluation (to report coverage after the fact) or during training (to compute rewards, gate trajectories, or build supervised targets); it restricts what the model reads at the moment it places guards. A solver allowed to query a visibility oracle at every decision step can in principle simulate greedy or local search, and a measurement of what such a solver has learned would be conflated with a measurement of what its oracle can compute. The geo-free constraint isolates the first question from the second, which is what makes "no explicit geometric knowledge" a measurable property and not just a rhetorical one.

3 Related Work

This section places the present work in three lines of research: classical algorithms for the Art Gallery Problem, neural combinatorial optimization with reinforcement learning, and supervised post-processing of learned solvers.

The Art Gallery Problem. Lee and Lin [10] established NP-hardness for several variants of the AGP; O'Rourke [12] surveys the foundational geometric and combinatorial results. The vertex-guard variant has a long history of approximation algorithms and exact integer-programming approaches; we treat published instance libraries [5] as the source of polygons in our experiments. Classical greedy heuristics [7] place guards to maximize marginal coverage and

are reliable on small to mid-sized polygons; local search refines candidate guard sets by swapping or removing vertices and is the standard high-quality baseline. We use these classical methods only as evaluation reference points and as a source of supervised labels for the representation probe in Section 4.3; they are not in competition with the reinforcement method on guard count.

Reinforcement learning for combinatorial optimization. Pointer networks [16] introduced the architecture of attending over input tokens to produce variable-length output index sequences, and were extended to combinatorial optimization with reinforcement learning by Bello et al. [1] using REINFORCE [17] with a learned baseline. Subsequent work introduced attention-only architectures for routing problems [8] and policy optimization variants such as POMO [9] that exploit problem symmetries. We adopt the more recent preference-optimization (PO) recipe of Pan et al. [13], which trains the policy with a Bradley–Terry [4] ranking loss over pairs of solutions sampled from the policy itself, replacing the variance-heavy REINFORCE objective. We treat PO as a reinforcement-style procedure: the loss is computed from policy rollouts on the environment (the polygon), the gradient direction is determined by the reward (coverage versus guard usage), and no supervised labels for the action sequence are used. A broader methodological survey is provided by Bengio et al. [2].

Supervised post-processing of learned solvers. An orthogonal line of work post-processes a base solver's output with a second model trained by supervised or imitation learning. For combinatorial problems with hard feasibility constraints (such as covering), iterative supervised editors must contend with compounding errors and a train–inference distribution gap; DAgger [14] and its variants partially address the latter. We use a single-shot supervised post-processor (the SetPredictor) not as a competing solver but as a *probe* of the RL-trained encoder's representation. The framing is methodological: if a small classifier reading frozen RL embeddings can recover feasibility, the RL encoder must already carry the relevant geometric information.

4 Method

We describe the reinforcement method (Section 4.1), the diagnostic that explains why a learned *iterative* editor over the same policy does not improve over it (Section 4.2), and the single-shot representation probe (Sections 4.3–4.5).

4.1 The reinforcement method: PO/BT pointer policy

Architecture. The policy is an LSTM-encoder pointer network with attention-based decoder, of roughly 364k parameters. A polygon s with vertex sequence $V(s) = (x_1, \ldots, x_n)$ is consumed coordinate-wise (no visibility input). The encoder produces per-vertex embeddings $\{h_i\}_{i=1}^{n}$, $h_i \in \mathbb{R}^{128}$. The decoder attends

over the encoder at each step, emits a vertex index or an EOS token, and stops at EOS or when the maximum length is reached. The decoder outputs a sequence $\pi = (\pi_1, \ldots, \pi_T)$; the resulting guard set is $S(\pi) = \{\pi_t : \pi_t \neq \text{EOS}\}$, and all metrics (coverage, $|S|/n$, $|S|/\text{OPT}$) are computed on $S(\pi)$.

Reward and rollouts. For each polygon s, we sample R rollouts from the policy and score each rollout by a scalar utility $u(\pi \mid s)$ that combines coverage and guard usage; the precise scalarization places coverage above a soft threshold and then prefers smaller guard sets. Coverage during training is computed using a discretized-visibility sampler (M=500 interior points per polygon) as a cheap approximation; CGAL exact visibility is reserved for evaluation.

Preference-optimization loss. For each pair (π^+, π^-) of rollouts with $u(\pi^+) > u(\pi^-)$ above a coverage gate, we apply the Bradley–Terry log-likelihood

$$\mathcal{L}_{\text{BT}}(\theta) \;=\; -\mathbb{E}_{(s,\pi^+,\pi^-)} \left[\log \sigma\big(\alpha \left[\log p_\theta(\pi^+|s) - \log p_\theta(\pi^-|s) \right]\big) \right], \qquad (3)$$

where $\log p_\theta$ is length-normalized over the index sequence (Pan et al. [13]'s length-control trick, since AGP solutions have variable length). The gradient direction is driven purely by which of the two rollouts achieved a higher reward; no supervised target sequence is used.

REINFORCE with a learned baseline is the classical choice for training pointer policies on combinatorial problems [1]. We adopted PO following Pan et al. [13], who report that binary preferences over pairs of rollouts retain useful gradient direction even when rewards saturate — the regime that REINFORCE with a value baseline struggles with on dense-reward routing problems. We did not run REINFORCE as a side-by-side baseline on vertex-guard AGP and rely on their analysis to motivate the choice; a measured comparison on AGP is left for future work. At the end of training we are left with a single deterministic policy that maps a polygon's coordinates to a guard sequence in one forward pass; this is the artifact whose properties the rest of the paper investigates, and from this point on its encoder is frozen and treated as a learned feature extractor. Figure 1 reports the held-out training dynamics across four reconstructed checkpoints and shows that coverage and cost move jointly over training rather than the optimization collapsing into a degenerate single-objective regime.

4.2 What the reinforcement method does and does not absorb

The reinforcement method ends up absorbing two qualitatively different parts of the problem in qualitatively different ways. On the cardinality axis it is broadly successful: on held-out in-distribution polygons the policy uses mean $|S|/n = 0.166$ vertices, comparable to greedy (0.152) and local search (0.137) at preserved coverage (Table 2), and it does so without ever being shown a visibility object at inference. On the feasibility axis it is less successful: 71/367 `dev_test` polygons end up under $\text{Cov}(S) = 0.95$, and on OOD `test` the fraction

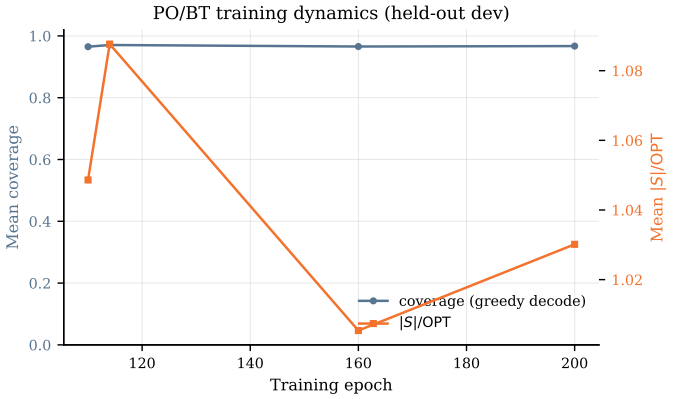

Figure 1: PO/BT training dynamics on the held-out `dev` split, reconstructed post-hoc from four saved checkpoints (see Section 8). Mean per-polygon coverage (left axis) and the approximation ratio $|S|/\mathrm{OPT}$ (right axis) of the greedy-decoded policy. The trace covers the late phase of training; the coverage–cost trade-off improves jointly rather than collapsing into a single-objective regime.

grows to 310/2107 with worst-polygon coverage near zero (Section 6.4). The reinforcement method has learned to use a small number of vertices, but it has not learned to guarantee that those vertices cover the polygon.

The natural reaction to this is to train a second learned model that applies a sequence of ADD/REMOVE/STOP edits to the policy's seed. We tried three such iterative editor architectures of increasing capacity — including one with topology features and an auxiliary visibility-prediction loss, and one with DAgger refresh [14] — and none of them improved over the pretrained policy in any coverage-preserving regime: in a representative held-out evaluation (30 polygons) the best editor variant reduced $|S|$ by about 24% relative to the seed but lost 0.005 in mean coverage and dropped to 0.27 *recovery rate* against the local-search reference set, well below the threshold where the editor could replace any baseline. Three structural reasons make iteration difficult here: rollouts accumulate errors because the editor sees clean trajectories at training but its own evolving predictions at inference; a single classification head over $\{\textsc{add}, \textsc{remove}, \textsc{stop}\} \times \{1, \ldots, n\}$ couples action selection with termination, producing STOP probabilities that are either too eager or too reluctant; and the training-time and inference-time state distributions diverge in ways that DAgger refresh alone did not close. We mention this only to motivate the single-shot design used for the representation probe: there is no rollout, there is no STOP token, and the model is trained and used in the same regime.

4.3 Probing the representation: a single-shot per-vertex classifier

The hypothesis under test asks what the reinforcement method has internalized. A natural way to test this is to ask whether a small classifier, conditioned only on the encoder's output and never on a visibility object at inference, can recover the feasibility decision the decoder failed to make. We call this classifier the *SetPredictor*, and we present it primarily as a representation diagnostic; its threshold sweep in Section 6 makes the diagnostic concrete on per-polygon evaluation, but the operating-curve framing that follows is a side-effect of the measurement, not a deployment recipe.

For polygon s with vertex sequence $V(s) = (x_1, \ldots, x_n)$, the input feature for vertex i is

$$z_i \;=\; [\, h_i \,;\, x_i \,;\, \mathbf{1}\{i \in \pi_{\text{seed}}\}\,] \;\in\; \mathbb{R}^{131}, \tag{4}$$

that is, the *frozen* 128-dimensional encoder embedding h_i from the PO/BT policy, the 2D coordinates x_i, and a single binary indicator for whether vertex i is selected by the policy's greedy-decoded seed π_{seed}. No visibility matrix, no CGAL output, and no area feature enters the classifier at inference time. The geo-free constraint is preserved.

The architecture is a small Transformer encoder over the n vertex tokens: 3 self-attention layers, 8 heads, hidden width 128, FFN expansion factor 4, and a sigmoid output head per token. Total parameters: 464,001 (henceforth $\sim$ 464k). We form a per-vertex representation by concatenating the attention output with two pooled context vectors — mean-pooled over the seed set and mean-pooled over all valid (non-padded) vertices — and feed the concatenation to a 2-layer MLP head:

$$\hat{p}_i \;=\; \sigma\big(\mathrm{MLP}([\, v_i \,\|\, S_{\text{pool}} \,\|\, G_{\text{pool}}\,])_i\big), \qquad i = 1, \ldots, n. \tag{5}$$

The output $\hat{p}_i \in [0, 1]$ is interpreted as the probability that vertex i should be in the final guard set. Thresholding at $t \in (0, 1)$ yields the guard set

$$S(t) \;=\; \{\, i \in \{1, \ldots, n\} : \hat{p}_i \geq t \,\}. \tag{6}$$

The SetPredictor is trained by supervised learning on labels derived from local search over the policy's seed (Section 4.4), so it is not itself a reinforcement learner; we use it as a probe of the RL-trained encoder. The only polygon-structural information it reads at inference is the frozen RL embedding h_i together with x_i and the seed indicator, and the logic of the probe is direct: if a small classifier reading only those embeddings can close the feasibility tail, the encoder must already carry the geometric information needed to support that decision.

4.4 Training the probe

Target sets. For each training polygon s we obtain a target set $\pi^{\star}_{\text{LS}}(s)$ by applying local search to the policy's seed solution. The local search uses a

coverage gate $\tau = 0.99$ relative to a reference computed with disc-vis sampling ($M = 500$), which is the cheap approximation we use at training time. CGAL exact visibility is used only at evaluation.

Loss. Binary cross-entropy per vertex with a positive-class weight that automatically balances the empirical guard/non-guard ratio (mean ≈ 5.66 on the development split),

$$\mathcal{L}(\theta) \;=\; -\frac{1}{|\mathcal{B}|}\sum_{s\in\mathcal{B}}\frac{1}{|V(s)|}\sum_{i=1}^{|V(s)|}\big[\, w_{+} y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\,\big], \qquad (7)$$

where $y_i = \mathbf{1}\{i \in \pi^{\star}_{\mathrm{LS}}(s)\}$ and w_+ is the positive-class weight.

Hyperparameters. 60 epochs, batch size 32, learning rate 3×10^{-4}, AdamW [11] with default weight decay. For the headline probe we train four independent runs with random seeds $\{1234, 11, 22, 33\}$ to report seed-variance (Table 2 and Section 6.4). The coordinate-only ablation (Section 6.2) and the figures (Sections 6.3, 6.4, 6.6) use seed 1234 as the displayed run. Checkpoint selection is on the `dev_tune` split (Section 5).

4.5 Inference and the threshold knob

At inference time the procedure is: run the PO/BT policy's encoder on the polygon's coordinates (one forward pass), greedy-decode a seed π_{seed}, form the per-vertex features z_i of Eq. (4), run one forward pass of the SetPredictor to obtain $\{\hat{p}_i\}_{i=1}^{n}$, and return $S(t)$ from Eq. (6). No CGAL call, no visibility matrix, no oracle anywhere on this path. The threshold t is the single continuous knob that controls the coverage–size trade-off: lower t admits more vertices, raising both expected coverage and $|S|$, and we report the full Pareto curve over $t \in \{0.20, 0.25, 0.30\}$ in Section 6.5. One could in principle iterate the SetPredictor by re-running it with the previous prediction's in-set indicator updated, for K passes; we evaluated $K \in \{1, 2, 3, 5\}$ over a sweep of thresholds and found that the predictions are empirically a fixed point after the first pass (Section 6.6). This is the expected behavior of a single-shot model whose input already includes the seed indicator, so we use $K = 1$ throughout.

5 Experiments

5.1 Dataset and partition

We draw polygons from the publicly available AGP instance library of Couto, de Rezende, and de Souza [5], which contains four families (random simple, random orthogonal, random von Koch, complete von Koch) over a wide range of vertex counts n. The library distributes *point-guard* optimal solutions — guards may be placed at any point of the polygon, not only at vertices —

Split	# polygons	n range	Use
train	9,791	8–150	Pointer + SetPredictor training
dev_tune	857	8–150	SetPredictor checkpoint selection
dev_test	367	8–150	Held-out evaluation (in-distribution)
test	2,107	12–1,000	Out-of-distribution evaluation
large	20	1,000–2,250	Extreme OOD (smoke)

Table 1: Dataset partition. The `dev` split is partitioned 70/30 into `dev_tune` (checkpoint selection) and `dev_test` (held-out evaluation) by seeded random shuffle (seed 1234). The `test` split contains polygons whose vertex count exceeds the training range and is used as the out-of-distribution evaluation. The `large` split is reserved for an extreme-OOD smoke evaluation.

which are not the optimum for the vertex-guard variant we study. We therefore compute vertex-guard optima OPT ourselves, by solving the vertex-guard set-cover integer program for each polygon, and use these computed OPT values throughout the paper. The splits we work with are listed in Table 1.

The 70/30 partition is critical: an earlier alphabetical-name split caused systematic family imbalance between `dev_tune` and `dev_test` (`fat-*` polygons concentrated in tune, `randsimple-*` in test), which produced misleading optimism. The seeded random shuffle removes this bias.

5.2 Training setup

The PO/BT reinforcement-trained policy is trained once and frozen; all coverage rewards during PO training use disc-vis sampling. The SetPredictor probe is trained on `train` only; `dev_tune` is used for early stopping and threshold inspection; `dev_test`, `test`, and `large` are touched only after training and threshold selection are complete. Hyperparameters for the SetPredictor are stated in Section 4.4; the policy itself is the existing `lstm_bt` checkpoint produced by our PO/BT training script.

5.3 Evaluation protocol

For each polygon we compute the policy's seed (the policy's own guard sequence, EOS-truncated and greedy-decoded), evaluate it with CGAL exact polygon visibility [15], and then, separately, run a single SetPredictor forward pass on the policy's encoder output, threshold at t to obtain $S(t)$, and evaluate $S(t)$ with CGAL. We report the metrics defined in Section 2.3: mean coverage $\mathrm{Cov}(S)$, mean normalized size $|S|/n$, mean approximation ratio $|S|/\mathrm{OPT}$, and the per-polygon coverage counts $\#\{\mathrm{Cov}(S) \geq c\}$ for $c \in \{0.95, 0.99, 0.999, 1\}$, plus the minimum coverage. Wilson 95% confidence intervals are reported on the proportion $\#\{\mathrm{Cov}(S) \geq 0.95\}/N$.

As noted in Section 5.1, vertex-guard optima OPT are not provided by the public library (which distributes point-guard solutions) and are computed by

us via integer programming. Our OPT values are available for all polygons in `train`, `dev`, and `test`, and unavailable for some polygons in `large` (where the ILP was too slow to terminate within our compute budget); we report coverage only on those.

6 Results

We organize the results around the hypothesis. Section 6.1 reports what the reinforcement method achieves on its own. Section 6.3 examines the per-polygon distributions behind the means. Section 6.4 is the main out-of-distribution measurement — the most informative test of whether the reinforcement-learned representation generalizes. Sections 6.5 and 6.6 report the cost of feasibility and the probe's fixed-point property.

6.1 The reinforcement method places guards

The third row of Table 2 is the geo-free reinforcement policy on its own: mean coverage 0.969, 296/367 polygons with $\mathrm{Cov}(S) \geq 0.95$ (Wilson 95% CI $(0.763, 0.844)$), and $|S|/\mathrm{OPT} \approx 1.09$. Read this against the question we asked in the introduction. A policy that sees only polygon coordinates and is trained from coverage-aware rewards over its own rollouts produces, on a held-out partition, guard sets that are competitive in cardinality with the classical anchors — mean $|S|/n = 0.166$ versus 0.152 for greedy and 0.137 for local search (Table 2, top block) — and reach 0.95 coverage on the large majority of polygons. The hypothesis — that vertex-guard placement is learnable, to a degree, by a reinforcement method that never reads an explicit visibility object at inference — is confirmed in the affirmative on this measurement.

The honest scope of the confirmation is the residual: $71/367 = 19.3\%$ of polygons fall below $\mathrm{Cov}(S) = 0.95$ under the policy alone (the distribution behind this is Section 6.3). The reinforcement method, on its own, does not give a feasibility guarantee. Figure 2 draws two illustrative polygons, one from `dev_test` and one from `test`, with the policy's seed, the policy plus the SetPredictor probe at $t = 0.20$ (the headline operating point of Table 2), and the optimum drawn side by side; it gives a visual sense of where the policy succeeds and where the probe contributes.

6.2 Does the encoder carry signal beyond coordinates?

The full SetPredictor receives three inputs per vertex: the frozen 128-dimensional encoder embedding h_i, the 2D coordinates x_i, and the seed-membership indicator. We isolate what the encoder contributes by retraining the probe with h_i masked to zero on every forward pass — architecture, parameter count (464,001), optimizer, and training schedule are identical.

On `dev_test` at $t = 0.20$ the no-encoder probe achieves mean coverage 0.995 versus the full probe's 0.998 ± 0.001 (mean $\pm$ std across four probe seeds, Ta-

Method	Mean cov	#{Cov ≥ .95}/N	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert$/OPT
Classical baselines				
Greedy	0.9994	367/367	0.1523	1.0086
Local search	0.9948	367/367	0.1367	0.8840[†]
Learned policy / probe				
Pretrained pointer (seed)	0.9689	296/367 (0.763,0.844)	0.1664	1.0878
SetPredictor full ($t = 0.20$)	**0.9984** ± 0.0011	366.2 ± 1.3/367	0.3890 ± 0.0606	2.5593 ± 0.4043
SetPredictor, no-encoder ($t = 0.20$)	0.9952	367/367	0.3978	2.5783

Table 2: Held-out evaluation on `dev_test` (367 polygons). Classical greedy and local search are the standard non-learned anchors (greedy guard set from [12]-style farthest-coverage; LS is the trajectory used to generate SetPredictor targets). The *seed* row is the PO/BT-trained policy decoded greedily. *SetPredictor full* is the headline representation probe; numbers are mean ± std across four probe seeds $\{1234, 11, 22, 33\}$ at fixed threshold $t = 0.20$. *SetPredictor, no-encoder* has the same architecture and parameter count with the frozen encoder masked to zeros (single-seed contrast, Section 6.2). Wilson 95% CI is shown for the policy seed only, where the feasibility proportion is informative; rows that reach 367/367 omit the degenerate CI.
[†]Local search reports $\lvert S\rvert/\text{OPT} < 1$ because the OPT reference is a time-limited B&B lower bound, not a verified optimum (Section 5); $\lvert S\rvert/n$ is the cardinality metric without this caveat.

ble 2). The per-polygon counts under a stricter $\text{Cov}(S) \geq 0.99$ feasibility gate diverge sharply: the full probe leaves between 4 and 27 polygons depending on seed (mean 11.3), while the no-encoder ablation leaves 57. On the OOD `test` split the gap is qualitative: the full probe averages 17.5 ± 12.5 sub-0.95 polygons (range 3–34 across seeds; Section 6.4), while the no-encoder probe leaves 81 at the same threshold and inherits the policy seed's worst-case coverage near zero. This is the direct measurement behind C1: the frozen encoder carries geometric structure that raw (x, y) alone does not reproduce, even when the rest of the decoder retains the full parameter budget to exploit it.

6.3 Per-polygon distributions behind the means

The mean coverage of the policy alone hides a bimodal distribution. Table 3 reports the per-polygon counts on `dev_test`: the policy reaches $\text{Cov}(S) = 1$ on only 3/367 polygons and $\text{Cov}(S) \geq 0.99$ on 99/367, with a worst polygon at 0.830. The probe at $t = 0.20$ eliminates the left tail and moves the bulk of the distribution above 0.99. Figure 3 gives the full picture: the coverage CDF (left column) confirms the SetPredictor curve sits to the right of the policy at every threshold, while the $\lvert S\rvert/n$ and $\lvert S\rvert$/OPT box plots (center and right) show the corresponding cost — the probe's guard sets are larger and more dispersed than the policy's seed. The top row is in-distribution `dev_test`; the bottom row is the OOD `test` split discussed next.

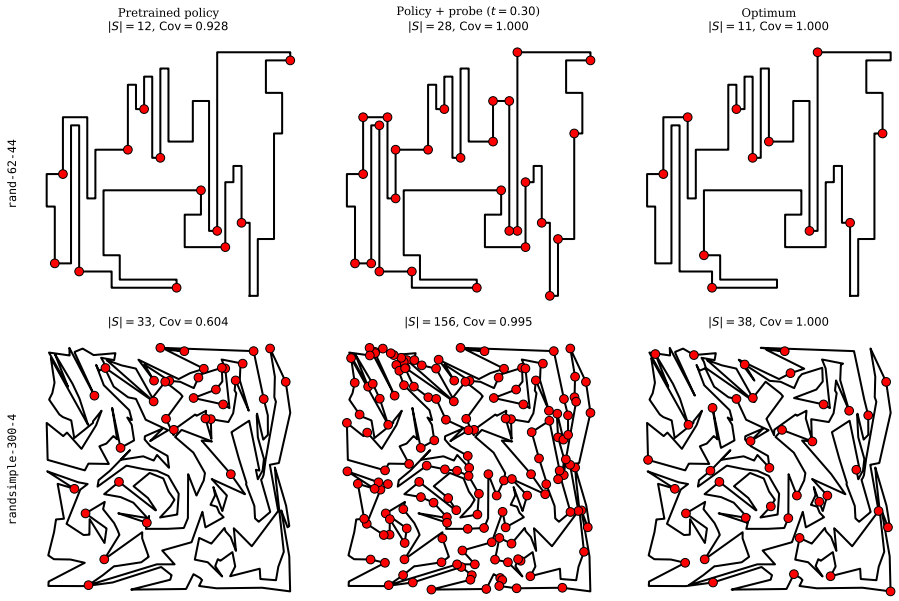

Figure 2: Worked examples: two polygons drawn with the policy's seed (left column), the policy plus the SetPredictor probe at $t = 0.20$ (middle column), and the optimum (right column). Vertices selected as guards are marked. Row 1 is an in-distribution polygon where the policy already covers most of the region. Row 2 is an out-of-distribution polygon where the policy's seed leaves a substantial uncovered region and the probe restores feasibility by adding vertices the policy missed.

6.4 Out-of-distribution generalization

The strongest test of what the reinforcement-trained representation carries is the OOD `test` split, whose polygons reach $n = 1000$ — roughly seven times the largest training polygon. The policy alone falls back to mean coverage 0.957 with 310/2107 polygons below 0.95 and a worst case of 0.049. The probe at $t = 0.20$, averaged across four seeds $\{1234, 11, 22, 33\}$, brings sub-0.95 failures down to 17.5 ± 12.5 of 2107 — a reduction of roughly an order of magnitude relative to the policy. The spread is informative: at fixed threshold $t = 0.20$ the four seeds land at {3, 8, 25, 34} sub-0.95 polygons, because each seed's best dev-tune checkpoint sits at a slightly different operating point on the threshold–cardinality curve (mean $|S|/n = 0.389 \pm 0.061$ across seeds). What is stable across seeds is the qualitative reduction; what is variable is exactly where on the Pareto each particular seed lands. The worst-polygon coverage at $t = 0.20$ ranges from 0.576 (seed 11) to 0.884 (seed 33); the displayed seed (1234) lifts it from the policy's worst case to 0.765. This is the evidence behind C2: a small classifier reading the frozen encoder's embeddings recovers feasibility on

Method	#{Cov = 1}	#{Cov ≥ .999}	#{Cov ≥ .99}	#{Cov ≥ .95}	min Cov
Pretrained pointer (seed)	3	15	99	296/367	0.830
SetPredictor $t = 0.20$	243	315	363	367/367	0.977
SetPredictor $t = 0.25$	195	283	361	367/367	0.962
SetPredictor $t = 0.30$	152	221	355	367/367	0.952

Table 3: Per-polygon coverage distribution on `dev_test` (367 polygons). Counts show the number of polygons reaching each coverage threshold; *min* Cov(S) is the worst polygon. The pretrained policy has a long left tail; the SetPredictor moves the entire distribution to the right. SetPredictor rows show the displayed seed (1234); Table 2 reports the four-seed mean±std on the same split at $t = 0.20$.

polygons far larger than any the encoder was trained on. Figure 4 disaggregates by polygon size, and the bottom row of Figure 3 shows the matching coverage and cost distributions.

The OOD failures at $t = 0.20$ are informative about where the probe's generalization breaks, and the failure mode itself is seed-dependent. For the displayed seed (1234) the three failures are not large polygons — they are the smallest in the OOD split (`rand-10-8` at $n = 10$, `rand-20-20` and `random-20-10` at $n = 20$), all with OPT $\in \{2, 3, 4\}$; on these tight instances the probe selects $|S| \in \{2, 4\}$ vertices and either matches the policy's seed coverage exactly or improves it by a single vertex but still falls short of 0.95. Across the other three seeds the pattern inverts: the failures concentrate at $n \geq 300$ (median failure-polygon $n = 425$ for seed 11, 50 for seed 22, 425 for seed 33) — the large-polygon end of OOD — so at-scale generalization is partially seed-dependent. Mean coverage at $n \geq 400$ remains strong across seeds (0.972–0.994 in the 401–500 bin) but the failure-rate variance reflects this scale-end sensitivity. We read this as: the encoder's bulk geometric structure generalizes (the four-seed mean failure count is one order of magnitude below the policy's 310, and mean coverage stays above 0.97 across the full n range), while the precise failure threshold along the size axis depends on which probe seed sets the calibration.

6.5 The cost of feasibility

The threshold t is the single knob trading coverage for guard count. At $t = 0.20$ three of the four probe seeds reach 367/367 on `dev_test` and the fourth leaves three (mean 0.75 ± 1.3 sub-0.95 polygons); $t = 0.20$ is also the most conservative operating point, with the largest $|S|$ and the fewest residual sub-0.99 polygons. The cost, measured against the optimum on `dev_test` as the four-seed mean, rises from $|S|/\text{OPT} \approx 1.09$ for the policy alone to $|S|/\text{OPT} = 2.02 \pm 0.21$ at $t = 0.30$ and 2.56 ± 0.40 at $t = 0.20$ — on the aggressive end the displayed seed reaches 2.20 and 2.90 respectively (Figure 3, $|S|/\text{OPT}$ column). The probe trades guard count for the missing coverage; we report the trade rather than fix a single operating point.

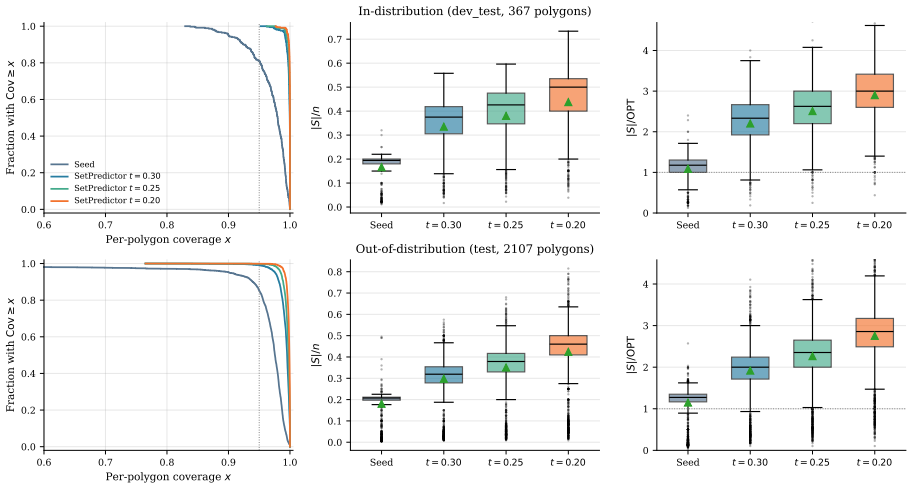

Figure 3: Distributional view of method performance, in-distribution `dev_test` (top, 367 polygons) and out-of-distribution `test` (bottom, 2107 polygons). *Left:* per-polygon coverage as a complementary stair-step CDF (fraction of polygons with coverage at least x); the dashed line marks the 0.95 feasibility gate. *Center, right:* box plots of the normalized guard count $|S|/n$ and the approximation ratio $|S|/\mathrm{OPT}$ for the policy seed and the three probe thresholds. The probe buys coverage at a controlled, visible cost in guard count. Boxes show the displayed seed (1234); Table 2 reports the four-seed mean±std at $t = 0.20$.

6.6 Iterative inference is a fixed point

Re-running the probe with its own output as the new in-set indicator does not change the prediction. Table 4 shows that across $K \in \{1, 2, 3, 5\}$ the changes in mean coverage, $|S|/n$, and $|S|/\mathrm{OPT}$ are at the fourth decimal place; Figure 5(a) plots the same across six thresholds $t \in \{0.5, 0.6, 0.65, 0.7, 0.75, 0.8\}$, visibly flat. This fixed-point property (C4) is the expected behavior of a single-shot model whose input already includes the seed indicator, and it is what distinguishes the probe from the iterative editors of Section 4.2: there is no stop time to calibrate and no rollout to drift. We use $K = 1$ throughout.

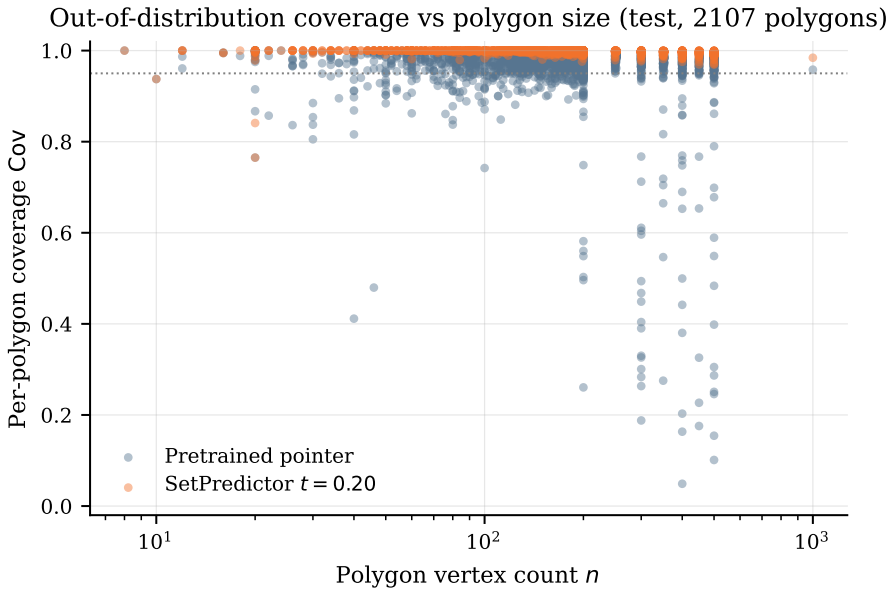

Figure 4: Per-polygon coverage as a function of vertex count n on the OOD `test` split ($n \in [12, 1000]$). The pretrained policy's coverage degrades systematically as n grows; the SetPredictor probe at $t = 0.20$ stays near the 0.95 feasibility line across the entire size range. Shown for the displayed seed (1234); across the four probe seeds the worst-polygon coverage at $t = 0.20$ ranges from 0.576 to 0.884 (Section 6.4).

Inference passes K	Mean Cov	Mean $\lvert S\rvert/n$	Mean $\lvert S\rvert$/OPT
$K = 1$	0.9723	0.1968	1.2328
$K = 2$	0.9712	0.1966	1.2302
$K = 3$	0.9719	0.1966	1.2295
$K = 5$	0.9719	0.1966	1.2292

Table 4: Inference passes $K \in \{1, 2, 3, 5\}$ at fixed threshold $t = 0.65$, full `dev` split (1224 polygons). The variation across K is at the fourth decimal: iterating the SetPredictor returns essentially the same prediction. We report the K-sweep at $t \in \{0.5, 0.6, 0.65, 0.7, 0.75, 0.8\}$ rather than the headline range $\{0.20, 0.25, 0.30\}$ because the fixed-point property is structural — a consequence of the seed indicator already being one of the model's inputs — and we expect it to be most stringently tested near the decision boundary where small probability shifts could flip the most indicators; the headline thresholds admit more vertices and therefore involve smaller fractions of indicator flips between $K = 1$ and $K = 2$. Figure 5(a) shows the same K-invariance holds across all six thresholds in this range.

7 Discussion

The single most informative measurement in this paper is what happens when we read the encoder rather than the decoder. The SetPredictor probe sees only frozen encoder embeddings and the policy's own seed indicator — no visibility matrix, no CGAL output, no per-vertex area feature — yet it closes the in-distribution feasibility tail on the displayed seed (Table 2) and compresses the out-of-distribution feasibility tail by roughly an order of magnitude on average across four probe seeds (Section 6.4). Two readings are consistent with this. Either the encoder learned enough geometric structure that a small classifier over its output can recover feasibility on its own, or the decoder's EOS calibration was the bottleneck and the policy had already identified the right vertices but stopped too early. Both support the same conclusion about what the reinforcement method internalized: a representation that contains more of the relevant geometry than the policy's decoder expressed. Two further measurements quantify this. First, the coordinate-only ablation in Section 6.2 shows that masking the encoder embedding to zeros, with everything else held constant, leaves a probe that cannot match the full probe's coverage at matched cardinality on either split. Second, fitting a plain L_2-regularized logistic regression on the frozen per-vertex embeddings against the LS-derived guard-membership label achieves ROC-AUC $= 0.842 \pm 0.024$ (5-fold polygon-grouped cross-validation, 200 polygons, 12,268 vertices, mean $\pm$ std across folds; PR-AUC $= 0.580 \pm 0.048$ against a positive-class prior of 0.16). Figure 5(b) visualizes the same signal in a 2D projection.

Recovering feasibility has a price, and we report it rather than disguise it. The probe's guard sets grow to roughly two to three times the optimum as the threshold is lowered (Section 6.5), and the $|S|/\mathrm{OPT}$ box plots of Figure 3 show that the whole distribution widens, not just its mean. A deployment would pick one operating point on this curve; we leave it as a knob because the right point depends on how costly an extra guard is relative to an uncovered region.

We are not claiming a better AGP solver. Classical greedy reaches mean coverage 0.999 at $|S|/n = 0.152$ and $|S|/\mathrm{OPT} = 1.009$ on `dev_test` (Table 2); the probe at $t = 0.20$ reaches mean coverage ≥ 0.998 at $|S|/\mathrm{OPT} \approx 2.6$ across four seeds. The classical anchors win on cardinality at preserved coverage, and the policy and probe are not built to contest that. What we document is what is learnable by reinforcement when the model is denied geometric input at the moment of placing guards, and what part of that learning lives in the representation rather than the decoder. The contrast between the failed iterative editors (Section 4.2) and the single-shot probe is the cleanest expression of the point: removing the stop time and the rollout is exactly what lets the encoder's information surface.

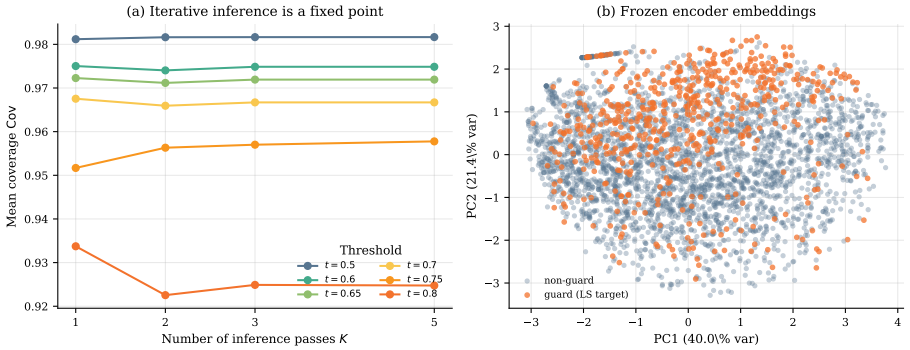

Figure 5: Two views of the probe's mechanism. *(a)* Iterative inference is a fixed point: mean coverage as a function of inference passes $K \in \{1, 2, 3, 5\}$ for six thresholds $t \in \{0.5, 0.6, 0.65, 0.7, 0.75, 0.8\}$, on the full `dev` split; the across-K variation is at the fourth decimal. *(b)* Two-dimensional projection of the frozen encoder's per-vertex embeddings, colored by guard-membership label (LS-derived target), pooled across held-out polygons. The separation visible here is corroborated quantitatively by a linear probe over the same 128-d embeddings (ROC-AUC 0.842 ± 0.024, Section 7); the 2D scatter is the visualization, the AUC is the measurement.

8 Limitations

1. **Scope of the "no geometric knowledge" claim.** The geo-free constraint is on inference: the policy and the probe see no visibility object at test time. *Training* does see visibility: the PO/BT reward is computed from disc-vis coverage, and the probe's supervised targets are generated by local search using disc-vis as a coverage gate. "No explicit geometric knowledge" must be read as "no geometric oracle at the moment of placing guards."

2. **The probe is supervised, not reinforcement.** The SetPredictor is trained by BCE against local-search-derived labels. It is not itself a reinforcement learner. We position it as a representation probe of the RL-trained encoder, not as an RL contribution.

3. **Held-out partition size.** `dev_test` has 367 polygons. Wilson 95% CIs on the coverage-feasibility proportion are reported, but the absolute count is modest. The OOD evaluation on `test` (2107 polygons) provides a much larger second held-out evidence base.

4. **Single-seed policy.** The PO/BT policy is trained with a single random seed (1234). We retrain the probe at four seeds $\{1234, 11, 22, 33\}$ and report mean $\pm$ std on `dev_test` (Table 2) and OOD `test` (Section 6.4); multi-seed policy training is left for future work.

5. **LSTM, not Transformer, encoder.** The reinforcement-trained policy is an LSTM pointer. A Transformer encoder might produce stronger embeddings and tighten the Pareto curve; we did not retrain.

6. **disc-vis approximation in training.** Both the PO/BT coverage reward and the SetPredictor training targets use disc-vis sampling at $M = 500$. Exact CGAL visibility is used only at evaluation.

7. **Extreme OOD unaddressed in the main results.** The `large` split (n up to 2250) is not evaluated in the main results table. A smoke run on 20 polygons confirms the probe still tracks the threshold curve; a full evaluation requires further engineering.

8. **Training-curve reconstruction.** Figure 1 is reconstructed from four saved checkpoints (epochs 110, 114, 160, 200), not from an online metric log; per-epoch coverage was not preserved during the original PO/BT run. The trace shows only late-training dynamics, which is sufficient to confirm that the gradient direction does not collapse but does not characterize early-training behavior.

9 Conclusion

We asked what a neural policy internalizes about vertex-guard placement when trained from a coverage-aware reward over its own rollouts and denied any geometric oracle at inference, and we answered it by reading the encoder rather than the decoder: a single-shot probe over the frozen embeddings closes the in-distribution feasibility tail on the displayed seed and compresses the out-of-distribution tail by roughly an order of magnitude across four probe seeds, which we take as evidence that the reinforcement-trained representation already carries more of the geometry than the policy's decoder revealed. The result is a measurement in a deliberately constrained setting, not a competitor to classical solvers on guard count — the learner is not the best AGP heuristic, but it arrived at a usable representation of vertex-guard placement without ever being shown the geometry it had to reason about at inference. Two extensions follow naturally: a stronger encoder, such as a Transformer trained under the same PO/BT objective, may shorten the residual tail directly and leave less work for the probe; and a per-instance threshold predicted from the same embeddings would replace the global knob with an adaptive one.

References

[1] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.

[2] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

[3] Bahram Sadeghi Bigham, S. Dolatikalan, and Alireza Khastan. Minimum landmarks for robot localization in orthogonal environments. *Evol. Intell.*, 15(3):2235–2238, 2022.

[4] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3–4):324–345, 1952.

[5] Marcelo C. Couto, Pedro J. de Rezende, and Cid C. de Souza. Instances for the Art Gallery Problem. `http://www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery`, 2009.

[6] A. Elnagar and L. Lulu. An art gallery-based approach to autonomous robot motion planning in global environments. In *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 2079–2084, 2005.

[7] Héctor González-Baños. A randomized art-gallery algorithm for sensor placement. *Proceedings of the 17th Annual Symposium on Computational Geometry (SoCG)*, pages 232–240, 2001.

[8] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*, 2019.

[9] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. POMO: Policy optimization with multiple optima for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

[10] D. T. Lee and A. K. Lin. Computational complexity of art gallery problems. *IEEE Transactions on Information Theory*, 32(2):276–282, 1986.

[11] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[12] Joseph O'Rourke. *Art Gallery Theorems and Algorithms*. Oxford University Press, 1987.

[13] Yu Pan et al. Preference optimization for neural combinatorial optimization. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[14] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011.

[15] The CGAL Project. CGAL user and reference manual. `https://doc.cgal.org/5.6/Manual/packages.html`, 2024.

[16] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

[17] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.